

TUGAS PERTEMUAN 10

Nama : Zaky Alnadif

NIM : J0403251126

1. Code Pemograman

```
2. # Membuat class Node
3. class Node:
4.     def __init__(self, data):
5.         self.left = None
6.         self.right = None
7.         self.data = data

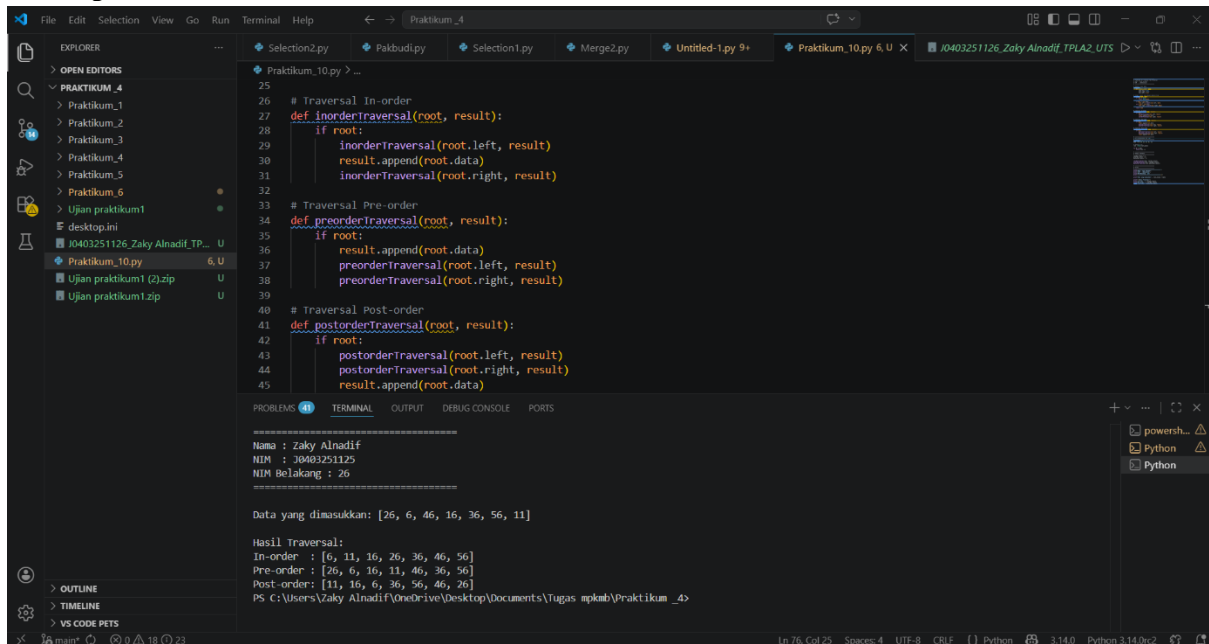
8. # Fungsi insert untuk Binary Search Tree
9. def insert(root, data):
10.     if root is None:
11.         return Node(data)
12.
13.     if data < root.data:
14.         root.left = insert(root.left, data)
15.     elif data > root.data:
16.         root.right = insert(root.right, data)
17.
18.     return root
19.
20. # Traversal In-order
21. def inorderTraversal(root, result):
22.     if root:
23.         inorderTraversal(root.left, result)
24.         result.append(root.data)
25.         inorderTraversal(root.right, result)
26.
27. # Traversal Pre-order
28. def preorderTraversal(root, result):
29.     if root:
30.         result.append(root.data)
31.         preorderTraversal(root.left, result)
32.         preorderTraversal(root.right, result)
33.
34. # Traversal Post-order
35. def postorderTraversal(root, result):
36.     if root:
37.         postorderTraversal(root.left, result)
38.         postorderTraversal(root.right, result)
39.         result.append(root.data)
40.
41. # =====
```

```

42. # DATA BERDASARKAN NIM (26)
43. # =====
44. root_value = 26
45. data = [6, 46, 16, 36, 56, 11]
46.
47. # Membuat tree
48. root = Node(root_value)
49.
50. for d in data:
51.     insert(root, d)
52.
53. # =====
54. # PROSES TRAVERSAL
55. # =====
56. inorder_result = []
57. preorder_result = []
58. postorder_result = []
59.
60. inorderTraversal(root, inorder_result)
61. preorderTraversal(root, preorder_result)
62. postorderTraversal(root, postorder_result)
63.
64. # =====
65. # OUTPUT
66. # =====
67. print("=====")
68. print("Nama : Zaky Alnadif")
69. print("NIM : J0403251125")
70. print("NIM Belakang : 26")
71. print("=====\n")
72.
73. print("Data yang dimasukkan:", [root_value] + data)
74.
75. print("\nHasil Traversal:")
76. print("In-order :", inorder_result)
77. print("Pre-order :", preorder_result)
78. print("Post-order:", postorder_result)

```

2. Output Dari Kode



The screenshot shows a VS Code editor with a Python file named 'Praktikum_10.py'. The code defines three traversal functions: `inordertraversal`, `preordertraversal`, and `postordertraversal`. The terminal output shows the execution of these functions with the input data [26, 6, 46, 16, 36, 56, 11].

```
25
26 # Traversal In-order
27 def inordertraversal(root, result):
28     if root:
29         inordertraversal(root.left, result)
30         result.append(root.data)
31         inordertraversal(root.right, result)
32
33 # Traversal Pre-order
34 def preordertraversal(root, result):
35     if root:
36         result.append(root.data)
37         preordertraversal(root.left, result)
38         preordertraversal(root.right, result)
39
40 # Traversal Post-order
41 def postordertraversal(root, result):
42     if root:
43         postordertraversal(root.left, result)
44         postordertraversal(root.right, result)
45         result.append(root.data)
```

Terminal Output:

```
=====
Nama : Zaky Alnadiif
NIM : 10403251126
NIM Belakang : 26
=====

Data yang dimasukkan: [26, 6, 46, 16, 36, 56, 11]

Hasil Traversal:
In-order : [6, 11, 16, 26, 36, 46, 56]
Pre-order : [26, 6, 16, 11, 46, 36, 56]
Post-order : [11, 16, 6, 36, 56, 46, 26]
PS C:\Users\Zaky Alnadiif\OneDrive\Desktop\Documents\Tugas mpkmb\Praktikum_4>
```

3. Hasil traversal:

- In-order : [6, 11, 16, 26, 36, 46, 56]
- Pre-order : [26, 6, 16, 11, 46, 36, 56]
- Post-order : [11, 16, 6, 36, 56, 46, 26]

4. Penjelasan singkat

Menurut saya perbedaan dari ketiga Traversal ini terletak pada urutan pengujungannya dan kegunaannya contoh In-order digunakan untuk menampilkan data secara terurut dari kecil ke besar. Pre-order digunakan untuk melihat struktur awal dari tree karena root ditampilkan terlebih dahulu. Sedangkan post-order digunakan untuk proses penghapusan atau evaluasi karena root dikunjungi terakhir. Nah menurut saya pre-order merupakan yang paling mudah digunakan untuk melihat struktur awal tree. Hasil traversal tidak akan sama apa bila urutan masuk data tidak akan sama karena struktur yang terbentuk bergantung pada urutan input data, jika struktur tree beda maka hasil nya juga akan beda tapi untuk In-order hasilnya akan tetap sama karena in-order hasil nya terurut

